CMPE 321 – Project 1: TransferDB
Part 2: Logical Database Design

Group 36: Halis Tunay Aslan, Kaan Uz

March 2026

1 Introduction

This document presents the logical database design for TransferDB, converting the ER diagrams from Part 1 into relational tables using MySQL DDL statements. All integrity constraints are specified using `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`, and `CHECK`. Constraints that cannot be enforced using basic MySQL syntax are documented in Section 3.

2 SQL DDL Statements

```sql
-- ============================================================
-- TransferDB     CMPE 321 Project 1, Part 2 (Logical Design)
-- MySQL DDL Statements
-- ============================================================

DROP DATABASE IF EXISTS TransferDB;
CREATE DATABASE TransferDB;
USE TransferDB;

-- Person (supertype)
CREATE TABLE Person (
    person_id INT,
    name VARCHAR(100) NOT NULL,
    surname VARCHAR(100) NOT NULL,
```

```
    nationality VARCHAR(100) NOT NULL,
    date_of_birth DATE NOT NULL,
    PRIMARY KEY (person_id)
);

-- Player ISA Person
CREATE TABLE Player (
    person_id INT,
    market_value DECIMAL(15,2) NOT NULL,
    main_position VARCHAR(20) NOT NULL,
    strong_foot VARCHAR(10) NOT NULL,
    height INT NOT NULL,
    PRIMARY KEY (person_id),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CHECK (market_value > 0),
    CHECK (height > 0),
    CHECK (main_position IN ('Goalkeeper','Defender','Midfielder','
       Forward')),
    CHECK (strong_foot IN ('Right','Left','Both'))
);

-- Manager ISA Person
CREATE TABLE Manager (
    person_id INT,
    preferred_formation VARCHAR(20) NOT NULL,
    experience_level VARCHAR(20) NOT NULL,
    PRIMARY KEY (person_id),
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Referee ISA Person
CREATE TABLE Referee (
    person_id INT,
    license_level VARCHAR(20) NOT NULL,
    years_of_experience INT NOT NULL,
    PRIMARY KEY (person_id),
```

```
    FOREIGN KEY (person_id) REFERENCES Person(person_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CHECK (years_of_experience >= 0)
);

-- ISA: covering + disjoint, so alternatively could use only 3
   tables (Player, Manager, Referee)
-- each containing Person attributes directly. We chose the general
   4-table approach (Person + subtypes)
-- to avoid attribute redundancy and allow querying all persons from
    a single table.
-- Cannot enforce: a Person must be exactly one of Player, Manager,
   or Referee (cross-table).

-- Stadium
CREATE TABLE Stadium (
    stadium_id INT,
    stadium_name VARCHAR(200) NOT NULL,
    city VARCHAR(100) NOT NULL,
    capacity INT NOT NULL,
    PRIMARY KEY (stadium_id),
    UNIQUE (stadium_name, city),
    CHECK (capacity > 0)
);

-- Club
-- "leads" relationship merged (manager_id), "primary_venue"
   relationship merged (stadium_id)
CREATE TABLE Club (
    club_id INT,
    club_name VARCHAR(200) NOT NULL,
    city VARCHAR(100) NOT NULL,
    foundation_year INT,
    manager_id INT NOT NULL,
    stadium_id INT NOT NULL,
    PRIMARY KEY (club_id),
    UNIQUE (club_name),
    UNIQUE (manager_id),
```

```
    FOREIGN KEY (manager_id) REFERENCES Manager(person_id),
    FOREIGN KEY (stadium_id) REFERENCES Stadium(stadium_id)
);
-- manager_id NOT NULL: total participation in "leads" (every club
   must have a manager)
-- UNIQUE(manager_id): key constraint in "leads" (a manager manages
   at most one club)
-- Cannot enforce: the squad of a club (set of players currently
   under contract) is derived from active contracts

-- Contract
-- "signs" (Player) and "with" (Club) relationships merged
-- History is preserved, records are never deleted
CREATE TABLE Contract (
    contract_id INT,
    player_id INT NOT NULL,
    club_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    weekly_wage DECIMAL(12,2) NOT NULL,
    contract_type VARCHAR(20) NOT NULL,
    PRIMARY KEY (contract_id),
    FOREIGN KEY (player_id) REFERENCES Player(person_id),
    FOREIGN KEY (club_id) REFERENCES Club(club_id),
    CHECK (end_date > start_date),
    CHECK (weekly_wage > 0),
    CHECK (contract_type IN ('Permanent','Loan'))
);
-- Cannot enforce: a player's current club is derived from active
   contracts (start_date <= NOW <= end_date)
-- Cannot enforce: a player who has no active contract is a free
   agent (derived)
-- Cannot enforce: a player may have at most one active Permanent
   and one active Loan contract (cross-row)
-- Cannot enforce: a Loan contract requires an active Permanent
   contract with a different club (cross-table)

-- Transfer Record
```

```sql
-- "including" (Player), "from" (Club), "to" (Club) relationships
   merged
-- History is preserved, records are never deleted
CREATE TABLE Transfer_Record (
    transfer_id INT,
    player_id INT NOT NULL,
    from_club_id INT NOT NULL,
    to_club_id INT NOT NULL,
    transfer_date DATE NOT NULL,
    transfer_fee DECIMAL(15,2) NOT NULL,
    transfer_type VARCHAR(20) NOT NULL,
    PRIMARY KEY (transfer_id),
    FOREIGN KEY (player_id) REFERENCES Player(person_id),
    FOREIGN KEY (from_club_id) REFERENCES Club(club_id),
    FOREIGN KEY (to_club_id) REFERENCES Club(club_id),
    CHECK (from_club_id <> to_club_id),
    CHECK (transfer_fee >= 0),
    CHECK (
        (transfer_type = 'Free' AND transfer_fee = 0)
        OR (transfer_type IN ('Purchase','Loan') AND transfer_fee >
           0)
    )
);
-- Cannot enforce: for a Loan transfer, from_club must be the player
   's current parent club (cross-table)
-- Cannot enforce: a Loan transfer requires the player to have an
   active permanent contract (cross-table)

-- Competition
CREATE TABLE Competition (
    competition_id INT,
    name VARCHAR(200) NOT NULL,
    season VARCHAR(20) NOT NULL,
    country VARCHAR(100) NOT NULL,
    competition_type VARCHAR(20) NOT NULL,
    PRIMARY KEY (competition_id),
    UNIQUE (name, season)
);
```

```sql
-- Club_Competition ("participates" relationship, many-to-many)
CREATE TABLE Club_Competition (
    club_id INT,
    competition_id INT,
    PRIMARY KEY (club_id, competition_id),
    FOREIGN KEY (club_id) REFERENCES Club(club_id),
    FOREIGN KEY (competition_id) REFERENCES Competition(
       competition_id)
);

-- Match
-- "belongs_to" (Competition), "home" (Club), "away" (Club), "
   played_at" (Stadium), "officiates" (Referee) merged
CREATE TABLE `Match` (
    match_id INT,
    competition_id INT NOT NULL,
    home_club_id INT NOT NULL,
    away_club_id INT NOT NULL,
    stadium_id INT NOT NULL,
    referee_id INT NOT NULL,
    match_datetime DATETIME NOT NULL,
    attendance INT NOT NULL,
    home_goals INT NOT NULL,
    away_goals INT NOT NULL,
    PRIMARY KEY (match_id),
    FOREIGN KEY (competition_id) REFERENCES Competition(
       competition_id),
    FOREIGN KEY (home_club_id) REFERENCES Club(club_id),
    FOREIGN KEY (away_club_id) REFERENCES Club(club_id),
    FOREIGN KEY (stadium_id) REFERENCES Stadium(stadium_id),
    FOREIGN KEY (referee_id) REFERENCES Referee(person_id),
    CHECK (home_club_id <> away_club_id),
    CHECK (attendance >= 0),
    CHECK (home_goals >= 0),
    CHECK (away_goals >= 0)
);
```

```
-- Cannot enforce: attendance must not exceed stadium capacity (
   cross-table)
-- Cannot enforce: no two matches at the same stadium within 120
   minutes (cross-row)
-- Cannot enforce: a club cannot play two matches within 120 minutes
    (cross-row)
-- Cannot enforce: a referee cannot officiate two matches within 120
    minutes (cross-row)
-- Result (Home Win / Away Win / Draw) is derived from home_goals vs
    away_goals, not stored (by design)

-- Match Stats ("match_stats" relationship, Player <-> Match, many-
   to-many)
CREATE TABLE Match_Stats (
    player_id INT,
    match_id INT,
    club_id INT NOT NULL,
    is_starter BOOLEAN NOT NULL,
    minutes_played INT NOT NULL,
    position_in_match VARCHAR(10) NOT NULL,
    goals INT NOT NULL,
    assists INT NOT NULL,
    yellow_cards INT NOT NULL,
    red_cards BOOLEAN NOT NULL,
    rating DECIMAL(3,1) NOT NULL,
    PRIMARY KEY (player_id, match_id),
    FOREIGN KEY (player_id) REFERENCES Player(person_id),
    FOREIGN KEY (match_id) REFERENCES `Match`(match_id),
    FOREIGN KEY (club_id) REFERENCES Club(club_id),
    CHECK (minutes_played BETWEEN 0 AND 120),
    CHECK (goals >= 0),
    CHECK (assists >= 0),
    CHECK (yellow_cards BETWEEN 0 AND 2),
    CHECK (rating BETWEEN 1.0 AND 10.0)
);
-- Cannot enforce: total participation of Match in match_stats
-- Cannot enforce: max 11 starters per club per match
-- Cannot enforce: max 23 players per club per match squad
```

```
-- Cannot enforce: 2 yellow cards in a match = automatic red card (
   derived logic)
-- Cannot enforce: a player can only play for the home or away club,
    not both (cross-table)
-- Cannot enforce: red card suspends player for club's next match in
    same competition (derived)
-- Cannot enforce: 5 yellow cards accumulated in a season/
   competition = suspension (derived)
```

3 Constraints Not Enforced via DDL

The following constraints from the project description cannot be enforced using basic MySQL `CHECK` constraints (which do not support subqueries in MySQL), and would require triggers or application-level logic.

- **ISA Disjointness/Covering:** A Person must be exactly one of Player, Manager, or Referee. This requires cross-table validation. We could have done this via deleting the Person table and transferring its attributes to all subtypes but choose otherwise to avoid redundancy and achieve flexibility.
- **Active Contract Rules:** A player's current club is derived from active contracts (where start_date $\leq$ current date $\leq$ end_date). A player with no active contract is a free agent. These are temporal, derived concepts.
- **Concurrent Contract Constraint:** A player must not have two active contracts of the same type running concurrently. This requires cross-row date overlap checks.
- **Loan Contract Validity:** A Loan contract is only valid if the player holds an active Permanent contract with a different club. This requires cross-table temporal validation.
- **Club Squad:** The squad of a club (set of players currently under contract) is derived from active contracts.
- **Loan Transfer Validation:** For a Loan transfer, the from_club must be the player's current parent club and the player must have an active Permanent contract. These are cross-table constraints.
- **Attendance $\leq$ Capacity:** The attendance of a match must not exceed the capacity of the stadium. This is a cross-table constraint between Matches and Stadium.

- **120-Minute Scheduling (Stadium):** No two matches may be played at the same stadium if their start times are within 120 minutes of each other. This is a cross-row constraint.
- **120-Minute Scheduling (Club):** A club may not appear in two matches (as home or away) whose start times are within 120 minutes of each other. This is a cross-row constraint.
- **120-Minute Scheduling (Referee):** A referee may not be assigned to two matches whose start times are within 120 minutes of each other. This is a cross-row constraint.
- **Match Result:** The outcome (Home Win, Away Win, Draw) is derived from home_goals vs away_goals and is not stored as a separate attribute (by design).
- **Total Participation (match_stats):** Every match must have at least one player participating. Total participation in many-to-many relationships cannot be enforced via DDL unless also has a key constraint.
- **Max 11 Starters:** A maximum of 11 players may be starters for the same club in any given match. This constraint requires counting players and cannot be enforced directly using basic DDL.
- **Max 23 Squad:** Each club may register at most 23 players for a match squad. This constraint requires counting participating players and cannot be enforced directly using basic DDL.
- **Automatic Red Card:** A player who receives 2 yellow cards in a single match receives a red card automatically. This derived logic requires triggers.
- **Player Team Exclusivity:** A player cannot appear in a match as both a home team and away team participant. This requires cross-row validation.
- **Red Card Suspension:** A player who receives a red card is automatically suspended for the club's next match in the same competition. This requires querying historical data.
- **Yellow Card Accumulation:** A player who accumulates 5 yellow cards across a single season in a given competition is suspended for the club's next match. This requires knowing the active state of the database (DDL not enough).